

JavaScript Function and Function Expressions

A function is an independent block of code that performs a specific task, while a function expression is a way to store functions in variables.

JavaScript Function Syntax

A JavaScript function is defined with the function keyword, followed by a **name**, followed by parentheses ().

Function names can contain letters, digits, underscores, and dollar signs (same rules as variables).

The parentheses may include parameter names separated by commas:

(parameter1, parameter2, ...)

Example

```
<html>
```

```
<body>
```

```
<h1>JavaScript Functions</h1>
```

```
<p>Call a function which performs a calculation and returns the result:</p>
```

```
<p id="demo"></p>
```

```
<script>
```

```
function myFunction(p1, p2) {
```

```
  return p1 * p2;
```

```
}
```

```
let result = myFunction(4, 3);
```

```
document.getElementById("demo").innerHTML = result;
```

```
</script></body></html>
```

Note

Function **parameters** are listed inside the parentheses () in the function definition. Function **arguments** are the **values** received by the function when it is invoked. Inside the function, the arguments (the parameters) behave as local variables.

Function Return

When JavaScript reaches a return statement, the function will stop executing. If the function was invoked from a statement, JavaScript will "return" to execute the code after the invoking statement. Functions often compute a **return value**. The return value is "returned" back to the "caller":

Example

```
<html>

<body>

<h1>JavaScript Functions</h1>

<p>Call a function which performs a calculation and returns the result:</p> <p
id="demo"></p>

<script>

let x = myFunction(4, 3);

document.getElementById("demo").innerHTML = x;

function myFunction(a, b) {

  return a * b;

}

</script>

</body></html>
```

Functions Used as Variable Values

Functions can be used the same way as you use variables, in all types of formulas, assignments, and calculations.

Example

```
<html>

<body>

<h1>JavaScript Functions</h1>

<p>Using a function as a variable:</p>

<p id="demo"></p>

<script>

let text = "The temperature is " + toCelsius(77) + " Celsius.";
document.getElementById("demo").innerHTML = text;

function toCelsius(fahrenheit) {

  return (5/9) * (fahrenheit-32);

}

</script>

</body>

</html>
```

Example

```
<html>

<head>

<script>

function myFunction(x,y) {

    let z=x*y;

    document.getElementById("demo").innerHTML =z;

}

</script>

</head>

<body>

<h1>HTML DOM Events</h1>

<h2>The onclick Event</h2>

<p>The onclick event triggers a function when an element is clicked on.</p>

<p>Click to trigger a function that will output the result of two arguments passing from two Input Numbers:</p>

<input type="number" id="n" name="n">

<input type="number" id="n1" name="n1">

<button onclick="myFunction(n.value,n1.value)">Click me</button>

<p id="demo"></p>

</body></html>
```

JavaScript HTML DOM EventListener

The addEventListener() method attaches an event handler to the specified element.

The addEventListener() method attaches an event handler to an element without overwriting existing event handlers.

You can add many event handlers to one element. You can add many event handlers of the same type to one element, i.e two "click" events. You can add event listeners to any DOM object not only HTML elements.

i.e the window object.

The addEventListener() method makes it easier to control how the event reacts to bubbling.

When using the addEventListener() method, the JavaScript is separated from the HTML markup, for better readability and allows you to add event listeners even when you do not control the HTML markup.

You can easily remove an event listener by using the `removeEventListener()` method.

Syntax

`element.addEventListener(event, function, useCapture);`

The first parameter is the type of the event (like "click" or "mousedown" or any other [HTML DOM Event](#).) The second parameter is the function we want to call when the event occurs. The third parameter is a boolean value specifying whether to use event bubbling or event capturing. This parameter is optional.

Example

```
<html>

<body>

<h2>JavaScript addEventListener()</h2>

<p>This example uses the addEventListener() method to attach a click event to a button.</p> <button
id="myBtn">Try it</button>

<p id="demo"></p>

<script>

document.getElementById("myBtn").addEventListener("click", displayDate); function
displayDate() {

document.getElementById("demo").innerHTML = Date();

}
</script>

</body></html>
```

Example

```
<html>

<body>

<h2>JavaScript addEventListener()</h2>

<p>This example uses the addEventListener() method on the window object.</p> <p>Try
resizing this browser window to trigger the "resize" event handler.</p> <p
id="demo"></p>

<script>

window.addEventListener("resize", function(){

document.getElementById("demo").innerHTML = Math.random();
```

```
});  
</script>  
</body>  
</html>
```

Event Occurs When Belongs To [abort](#) The loading of a media is aborted [UiEvent](#), [Event](#)

[blur](#) An element loses focus [FocusEvent](#)

[canplay](#) The browser can start playing a media (has buffered [Event](#)
enough to begin)

[change](#) The content of a form element has changed [Event](#) [click](#) An element is clicked on

[MouseEvent](#) [dblclick](#) An element is double-clicked [MouseEvent](#) [drag](#) An element is being
dragged [DragEvent](#)

[drop](#) A dragged element is dropped on the target [DragEvent](#)

[fullscreenerror](#)

An element can not be displayed in fullscreen mode [Event](#)

[input](#) An element gets user input [InputEvent](#), [Event](#) [keydown](#) A key is down [KeyboardEvent](#)

[keypress](#) A key is pressed [KeyboardEvent](#) [keyup](#) A key is released [KeyboardEvent](#) [load](#) An object

has loaded [UiEvent](#), [Event](#) [mousedown](#) The mouse button is pressed over an element [MouseEvent](#)

[mouseenter](#) The pointer is moved onto an element [MouseEvent](#) [mouseleave](#) The pointer is moved

out of an element [MouseEvent mousemove](#) The pointer is moved over an element [MouseEvent](#)

[mouseover](#) The pointer is moved onto an element [MouseEvent mouseout](#) The pointer is moved out

of an element [MouseEvent mouseup](#) A user releases a mouse button over an element [MouseEvent](#)

Mousewheel

Deprecated. Use the [wheel](#) event instead [WheelEvent](#)

[offline](#) The browser starts working offline [Event online](#) The browser starts working

online [Event](#)

The removeEventListener() method

The removeEventListener() method removes event handlers that have been attached with the addEventListener() method:

Example

```
<html>

<head>

<style>

#myDIV {

background-color: coral;

border: 1px solid;

padding: 50px;

color: white;

font-size: 20px;

}

</style>

</head>

<body><h2>JavaScript removeEventListener()</h2>
```

```
<div id="myDIV">
```

```
<p>This div element has an onmousemove event handler that displays a random number every time you move your mouse inside this orange field.</p>
```

```
<p>Click the button to remove the div's event handler.</p>
```

```
<button onclick="removeHandler()" id="myBtn">Remove</button> </div>
```

```
<p id="demo"></p>
```

```
<script>
```

```
document.getElementById("myDIV").addEventListener("mousemove", myFunction);  
function myFunction() {
```

```
document.getElementById("demo").innerHTML = Math.random(); }
```

```
function removeHandler() {
```

```
document.getElementById("myDIV").removeEventListener("mousemove", myFunction); }
```

```
</script></body></html>
```